

TYPE ANNOTATIONS IN PYTHON

Make your code self-documenting • Catch bugs early



Code and Debug



Python for Beginners



Why Type Annotations?

Without types

```
1 def calculate(a, b):  
2     return a + b  
3  
4 calculate("5", 10)  
5 # TypeError: can only concatenate  
   str (not "int") to str
```



Python is dynamically typed, which is flexible but can cause runtime bugs in larger code.



Type annotations declare expected types directly in the function signature.

With types

```
1 def calculate(a: int, b: int) -> int:  
2     return a + b
```

Parameter
a expects
an `int`

Parameter
b expects
an `int`

Return
Function
returns an `int`



Clear contract. Fewer surprises.



This kind of mismatch causes many works-on-my-machine bugs.



Basic Type Hints



```
1 def add(x: int, y: int) -> int:
2     return x + y
3
4 def greet(name: str) -> str:
5     return f"Hello, {name}!"
6
7 def is_adult(age: int) -> bool:
8     return age >= 18
9
10 def divide(a: float, b: float) -> float:
11     return a / b
12
13
14
```

```
def add(x: int, y: int) -> int:
```

argument type

return type



Built-in Types

int

float

bool

str

bytes

None



QUICK TIP

Use `-> None` for functions that return nothing.

Variable Annotations



```
1 name: str = "Anirudh"
2 age: int = 22
3 is_active: bool = True
4 height: float = 5.9
5
6
7 score: int
8 score = 95
```



Self-documenting code



Better autocomplete



Type checkers catch mistakes early



Smarter IDE, fewer bugs.



QUICK TIP

Variable annotations shine most for class attributes and module-level constants.

Collection Types: List, Dict, Tuple, Set



List

`list[str]`

```
def get_names() -> list[str]:  
    return ["Anirudh", "Priya"]
```



Dict

`dict[str, int]`

```
def get_scores() -> dict[str, int]:  
    return {"Anirudh": 95, "Priya": 88}
```



Tuple

`tuple[int, int]`

```
def get_coords() -> tuple[int, int]:  
    return (10, 20)
```



Set

`set[str]`

```
def get_tags() -> set[str]:  
    return {"python", "coding"}
```



Modern syntax like `list[str]` is preferred in Python 3.9+, while older code may use `typing` module forms like `List[str]`, `Dict[str, int]`, `Tuple[int, int]`, and `Set[str]`.



Optional and Union

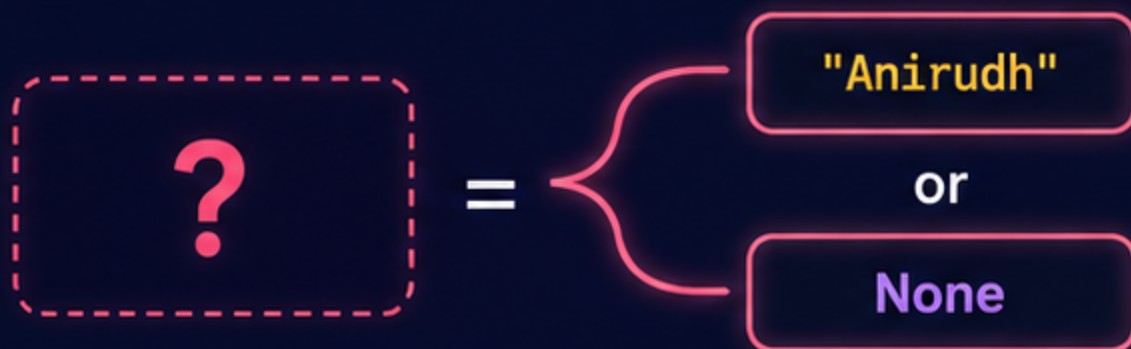


Optional

```
1 from typing import Optional
2
3 def find_user(user_id: int) -> Optional[str]:
4     if user_id == 1:
5         return "Anirudh"
6     return None
```

Modern syntax

```
def find_user(user_id: int) -> str | None: ...
```

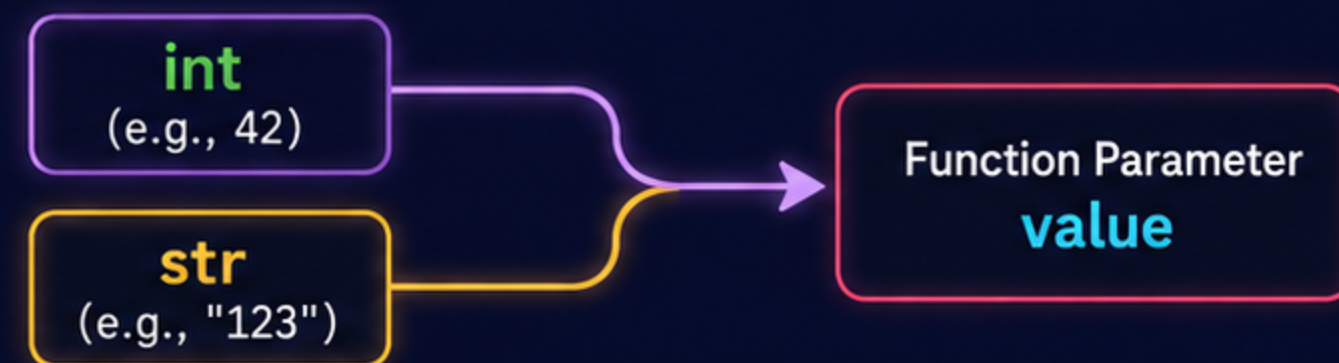


Union

```
1 from typing import Union
2
3 def parse(value: Union[int, str]) -> int:
4     return int(value)
```

Modern syntax

```
def parse(value: int | str) -> int: ...
```



QUICK TIP Optional[X] is shorthand for Union[X, None].



Optional = might be None. **Union** = could be many types.



Type Hints Don't Actually Enforce Types

Runtime

```
1 def add(x: int, y: int) -> int:  
2     return x + y  
3  
4 add("hello", "world")  
5 # returns "helloworld"
```



✓ no error



mypy / IDE

```
1 def add(x: int, y: int) -> int:  
2     return x + y  
3  
4 add("hello", "world")
```

❗ error: Argument 1 to 'add' has incompatible type 'str'; expected 'int'

```
>_ pip install mypy  
>_ mypy script.py
```



Python does not enforce type hints at runtime - they are informational. They become powerful when paired with tools like mypy and IDEs.



Type hints are a contract - Python won't enforce it for you.



Python won't catch it. mypy will.